

Notes de Cours :

CM du 16/01 :

Les architectures logicielles, les systèmes distribués, la conteneurisation et les protocoles de communication.

1. Concepts Fondamentaux du Développement

Framework vs Bibliothèque (Library)

- **Bibliothèque** : Une boîte à outils où le développeur pioche des fonctions. C'est le développeur qui a le contrôle et appelle le code.
- **Framework (Cadre de travail)** : Structure rigide qui impose une façon de faire.
 - *Objectif* : Automatiser les tâches répétitives et génériques pour se concentrer sur la logique métier.

Architecture Web

Une application web se divise généralement en deux parties :

1. **Backend (Serveur)** : Logique métier, accès aux données, souvent dans le Cloud.
2. **Frontend (Client)** : Interface utilisateur.

2. Architectures Logicielles

Monolithe vs Microservices

- **Monolithe** :
 - Un seul bloc de code déployé.
 - *Problème* : Si une partie tombe en panne, tout peut tomber. Difficile à maintenir et à faire évoluer (passage à l'échelle complexe).
- **Microservices** :
 - Découpage de l'application en petits services indépendants (ex: service de paiement, service d'affichage).
 - *Communication* : Via des messages légers (JSON/XML).
 - *Avantages* :
 - **Scalabilité horizontale** : On peut ajouter des machines pour gérer plus de charge sur un service précis.
 - **Tolérance aux pannes** : Si le service A plante, le service B continue de fonctionner.
 - **Interopérabilité** : Chaque service peut être écrit dans un langage différent.

Modèle Maître-Esclave (Master-Slave)

- Un nœud **Maître** distribue les tâches.
- Plusieurs nœuds **Esclaves** exécutent le travail.
- *Gestion de panne* : Si un esclave tombe, le maître redistribue. Si le maître tombe, un mécanisme d'élection désigne un nouveau maître.

3. Déploiement et Conteneurisation (Docker)

L'objectif est de résoudre le problème « ça marche sur ma machine mais pas en prod ».

- **Docker** : Permet de créer des conteneurs standardisés.
- **Conteneur vs Machine Virtuelle (VM)** :
 - Le conteneur partage le noyau (kernel) du système hôte (Linux) -> Plus léger.
 - La VM virtualise tout le matériel (OS complet) -> Plus lourd.

- **Dockerfile** : Fichier texte contenant les instructions pour construire l'image de l'application (le plan de construction).
-

4. Middleware et Communication Distribuée

L'objectif est de faire communiquer des objets distants (sur des machines différentes) de manière transparente.

RMI (Remote Method Invocation)

Technologie propre à **Java**.

- **Principe** : Permet d'invoquer une méthode sur un objet distant comme s'il était local.
- **Fonctionnement** :
 - **Stub (Client)** : Représente l'objet distant côté client. Il sérialise les paramètres.
 - **Skeleton (Serveur)** : Reçoit l'appel, désérialise, exécute la méthode réelle et renvoie le résultat.
 - **Registry** : Annuaire où les objets sont enregistrés pour être trouvés.
- **Limites** : Couplage fort (Java uniquement des deux côtés).

Evolution des technologies

1. **RMI** : Java only.
2. **CORBA** : Multi-langages mais très complexe.
3. **SOAP / REST** : Standards du Web (les plus utilisés aujourd'hui).

Comparatif Rapide

Technologie	Langage	Type	Caractéristique
RMI	Java	Objet Distant	Facile mais limité à Java
CORBA	Multilangue	Objet Distant	Complexe à mettre en œuvre
REST / SOAP	Tous	Web Service	Standard Web, Interopérable

5. Services Web : SOAP vs REST

Ce sont des moyens d'implémenter des API pour l'interopérabilité entre applications hétérogènes.

SOAP (Simple Object Access Protocol)

- **Nature** : Un **Protocole** strict.
- **Format** : Basé exclusivement sur **XML**.
- **Structure** : Utilise une enveloppe (Envelope) pour les messages.
- **Description** : Utilise le **WSDL** (Web Service Description Language) pour décrire les méthodes disponibles (le contrat).

REST (Representational State Transfer)

- **Nature** : Un **Style d'architecture** (pas un protocole).
- **Format** : Souvent JSON (plus léger), mais peut utiliser XML, TXT, etc.
- **Principe** : Tout est une **Ressource** identifiée par une **URL** (Uniform Resource Locator).
- **Verbes HTTP** : Utilise les méthodes standard du web pour agir sur les ressources :
 - **GET** : Lire / Récupérer
 - **POST** : Créer
 - **PUT** : Modifier / Remplacer
 - **DELETE** : Supprimer

Cycle de vie d'un appel (RMI/RPC)

1. Le **Client** appelle la méthode.
2. Le **Stub** sérialise (transforme en flux de données) les paramètres.
3. Transfert réseau.
4. Le **Skeleton** désérialise.
5. Le **Serveur** exécute le code.
6. Retour de la réponse (chemin inverse).

CM du 27/03 :

Introduction à Docker et la Conteneurisation. De manière générale, **Docker** est un outil de conteneurisation au cœur de la culture **DevOps**. Son but principal est de permettre le déploiement d'applications sans se soucier des versions ou des configurations de l'environnement cible.

Virtualisation Lourde (VM) vs Virtualisation Légère (Conteneur)

- **Virtualisation Classique (VM)** : Pour faire cohabiter plusieurs systèmes, chaque Machine Virtuelle nécessite son propre noyau (OS complet), ce qui la rend lourde et gourmande en ressources. Les processus sont séparés pour éviter qu'ils ne partagent le même espace mémoire.
- **Conteneurisation (Docker)** : C'est une virtualisation légère. Les conteneurs partagent le même système d'exploitation (le noyau hôte), qui est géré par le gestionnaire de conteneurs (Docker Engine).

Les 6 Avantages de la Conteneurisation

1. **Portabilité** : Plus besoin d'installer tout l'environnement (ex: JVM pour Java, Python, Flask) sur la machine hôte. Il suffit d'avoir Docker. L'image embarque toutes les dépendances.
2. **Isolation** : Chaque conteneur utilise ses propres ressources dans un environnement isolé, mais contrairement à une VM lourde, l'allocation d'espace mémoire ou de CPU est beaucoup plus souple.
3. **Haute Disponibilité & Démarrage rapide** : Démarrer une VM prend quelques minutes ; démarrer un conteneur prend quelques secondes. Si une application tombe en panne, le conteneur peut être redémarré presque instantanément.
4. **Facilitation du CI/CD (Intégration/Déploiement Continu)** : S'intègre parfaitement dans les pipelines Git (tests unitaires, tests de non-régression) avec des déploiements très rapides.
5. **Coût réduit & Efficacité** : Lié à sa légèreté, Docker permet de réduire les infrastructures matérielles nécessaires (contrairement aux VMs qui dupliquent les noyaux OS).
6. **Compatibilité Cloud** : Idéal pour le modèle économique du Cloud (*Pay as you go*). L'application s'exécute de manière transparente, peu importe le serveur sous-jacent.

Note historique : La conteneurisation existait déjà avec Linux (LXC), mais Docker a standardisé et simplifié son utilisation.

7. Concepts Clés : Images et Conteneurs

Image vs Conteneur

- **L'Image** : C'est un ensemble de couches (*layers*) en lecture seule qui contient l'application et toutes ses dépendances. On peut la comparer à une **Classe** en Java.
- **Le Conteneur** : C'est l'instance en cours d'exécution d'une image. On peut le comparer à un **Objet** instancié en Java. On peut lancer plusieurs conteneurs à partir d'une même image.

Commandes de base (CLI)

Bien qu'il existe une interface graphique (Docker Desktop), il est fortement conseillé de débiter et de travailler en ligne de commande.

Commande	Action
----------	--------

Commande

`docker --version`

`docker help`

`docker images` (ou `image ls`)

`docker ps`

`docker ps -a`

Action

Affiche la version installée.

Affiche l'aide (Raccourci Terminal : `Cmd + K` ou `clear` pour nettoyer).

Liste les images téléchargées en local.

Liste les conteneurs actuellement en cours d'exécution.

Liste **tous** les conteneurs (même ceux arrêtés).

Fonctionnement de `docker run`

Exemple : `docker run hello-world`

1. Docker cherche l'image en local.
2. S'il ne la trouve pas, il se connecte au registre public (**Docker Hub**).
3. Il télécharge (*pull*) l'image en local.
4. Il l'instancie et l'exécute. (Le premier lancement est plus long à cause du téléchargement. Les lancements suivants sont instantanés).

8. Mode Interactif et Manipulation d'Images

La plus petite image Linux fonctionnelle existante est **Alpine**.

1. Entrer dans un conteneur interactif

```
docker run -it alpine:latest /bin/sh
```

- `run` : Pour exécuter.
- `-it` : Interactive Terminal. Permet de garder le terminal ouvert et d'interagir.
- `/bin/sh` : Lance le shell à l'intérieur du conteneur.

Une fois à l'intérieur, on peut utiliser les commandes Linux classiques (`ls`, etc.). Par exemple, installer `curl` et faire une requête :

```
apk add --no-cache curl
curl http://google.com
```

2. Le problème de la persistance des données

Si on quitte le conteneur interactif précédent (`exit`), les modifications (comme l'installation de `curl`) sont perdues pour les prochaines instanciations. Le conteneur sert juste à emballer et exécuter ; il n'est **pas fait pour la persistance des données**. Pour sauvegarder des données (ex: Base de données), il faudra utiliser des **Volumes**.

3. Sauvegarder un conteneur modifié (Export / Import)

(Note : Cette manipulation est utile pour comprendre le fonctionnement sous le capot, mais en entreprise, on passe systématiquement par des *Dockerfiles* et des registres comme *Docker Hub*).

1. Trouver l'ID du conteneur : `docker ps -a` (les 4-5 premiers caractères suffisent).
2. Exporter le conteneur en archive `.tar` :

```
docker export <ID_CONTENEUR> -o myconteneur.tar
```

3. Importer l'archive pour en faire une nouvelle image :

```
docker import myconteneur.tar nouveau_nom:v1
```

(Si on ne précise pas :vl, Docker appliquera le tag :latest par défaut).

4. Lancer la nouvelle image :

```
docker run -it nouveau_nom:vl /bin/sh
```

4. Supprimer des Images et Conteneurs

- On ne peut pas supprimer une image si elle est utilisée par un conteneur (même arrêté).
- **Supprimer un conteneur** : `docker rm <ID_CONTENEUR>` (utiliser `docker ps -a` pour trouver l'ID).
- **Supprimer une image** : `docker rmi <NOM_IMAGE>`

9. Serveur Web, Mapping de Ports et Volumes (Nginx)

Sur une machine classique, on ne peut pas avoir deux applications qui écoutent sur le même port (ex: 8080). Avec Docker, c'est possible à l'intérieur des conteneurs, à condition de mapper les ports vers l'extérieur.

1. Lancer un serveur Nginx avec Mapping de port

```
docker pull alpine # Télécharge l'image en local
docker run -d -p 8080:80 --name monServeur nginx
```

- `-d (detached)` : Le conteneur démarre en arrière-plan et rend la main sur le terminal.
- `-p 8080:80` : Mappe le port 8080 de notre machine physique vers le port 80 du conteneur.
- `--name` : Permet de donner un nom personnalisé au conteneur (ici "monServeur") au lieu d'un nom généré aléatoirement.

En allant sur `http://localhost:8080/`, on voit la page par défaut "Welcome to nginx!".

2. Entrer dans un conteneur DÉJÀ en cours d'exécution

```
docker exec -it monServeur bash
```

Analogie : > * `docker run -d` a construit la maison (le conteneur) et vous a laissé à l'extérieur.

- `docker exec -it` c'est ouvrir la porte d'entrée et entrer dans la maison pour voir ce qui s'y passe, sans l'arrêter.

Une fois dedans, on peut modifier la page d'accueil :

```
cd /usr/share/nginx/html
apt-get update
apt-get install nano -y
nano index.html
```

3. Arrêter et gérer le cycle de vie

- `docker stop monServeur` : Arrête le conteneur. Le site n'est plus accessible.
- `docker restart monServeur` : Relance le conteneur. Les modifications faites avec `nano` sont toujours là car **le conteneur existe toujours**.
- `docker rm monServeur` : Détruit le conteneur. Si on relance un `docker run nginx`, on perd toutes nos modifications !

4. Les Volumes : Pour la vraie persistance

Pour que le code de notre site survive à la destruction du conteneur, on utilise un volume pour faire le lien entre notre dossier local et le dossier du conteneur.

```
# On s'assure d'être dans le dossier personnel
cd ~

# On crée un nouveau dossier pour le site et on rentre dedans
mkdir mon-site-persistent
cd mon-site-persistent

# On crée un fichier index.html personnalisé
echo "<body style='background: #eed9c4;'"><h1>Bienvenue sur mon site indestructible !</h1></body>" > index.html

# On lance le conteneur en liant le volume (-v)
docker run -d -p 8080:80 -v $(pwd):/usr/share/nginx/html --name monNouveauServeur nginx
```

- `-v $(pwd):/usr/share/nginx/html` : Dit à Docker "Prends le dossier courant (`pwd`) et monte-le à la place du dossier `/usr/share/nginx/html` du conteneur". Ainsi, si on modifie le fichier depuis notre machine ou si le conteneur est supprimé, les données restent en sécurité en local !

CM du 03/04 :

10. Persistance des Données : Les Volumes

Par défaut, un conteneur possède un système de fichiers éphémère. Si on supprime et redémarre un conteneur, toutes les modifications et données créées à l'intérieur sont détruites. Pour conserver ces données (ex: une base de données), il faut les externaliser sur la machine hôte.

Il existe 3 principales façons de gérer la persistance dans Docker :

1. Lancer un serveur Nginx avec Mapping de port

Bind Mounts (Montage de répertoires de l'hôte)

- **Principe** : On lie un répertoire spécifique et visible de la machine hôte directement dans le conteneur. Tout changement de l'un se reflète en temps réel dans l'autre.
- **Avantages** : Très flexible, idéal pour le développement (le code modifié sur la machine hôte est immédiatement pris en compte par le conteneur).
- **Inconvénients** : Dépendant du système de fichiers de l'hôte, peut poser des risques de sécurité si mal configuré.
- **Syntaxe** : On utilise le flag `-v` (qui remplace la logique du `-p` pour les ports) en précisant le chemin absolu de l'hôte vers le chemin du conteneur.

```
# Exemple : Tout changement dans le dossier projets se reflète dans /app
docker run -it -v /home/user/projets:/app alpine sh
```

2. Volumes gérés par Docker (Named Volumes)

- **Principe** : C'est Docker qui gère lui-même le stockage. Les données sont stockées dans une zone sécurisée de l'hôte (généralement `/var/lib/docker/volumes/`).
- **Avantages** : Indépendants du système de fichiers hôte, beaucoup plus sûrs, et faciles à sauvegarder/migrer. C'est la méthode recommandée en production.
- **Commandes utiles** :

`docker volume create monVolume` : Créer un volume.

`docker volume ls` : Lister les volumes existants.

`docker volume rm monVolume` : Supprimer un volume.

Exemple 1 : Partage de données entre deux conteneurs

```
# 1. Création du volume
docker volume create monVolume

# 2. Lancement d'un 1er conteneur avec ce volume et écriture d'un fichier
docker run -it -v monVolume:/data alpine sh
/ # echo "Bonjour tout le monde" >> /data/bjrtlm.txt
/ # exit

# 3. Lancement d'un 2ème conteneur sur le MÊME volume
docker run -it --name monDeuxiemeConteneur -v monVolume:/data alpine sh
/ # cat /data/bjrtlm.txt

# Résultat : Affiche "Bonjour tout le monde". Les données ont survécu et sont partagées !
```

Exemple 2 : Persister une base de données MySQL

```
docker volume create mysql_data

# Lancement de MySQL en liant le volume au dossier où MySQL stocke ses données en interne
docker run -d --name mysql -e MYSQL_ROOT_PASSWORD=pass -v mysql_data:/var/lib/mysql mysql:latest
```

3. Tmpfs (Stockage en mémoire RAM)

- **Principe** : Les données sont stockées temporairement dans la mémoire vive (RAM) et ne laissent aucune trace sur le disque dur.
- **Critères** : Dans la hiérarchie de la mémoire (Cache > RAM > Disque dur), plus on monte, plus les performances augmentent, mais le coût aussi.
- **Avantages** : Extrêmement rapide.
- **Cas d'usage** : Utile si la préoccupation première est la performance absolue et non la persistance (ex: Base de données "in-memory" comme H2, génération de liens valables 24h qui disparaissent au redémarrage).

Syntaxe :

```
docker run --tmpfs /chemin_dans_conteneur nom_image
```

11. Gestion des Réseaux (Networking)

Les réseaux permettent aux conteneurs de communiquer entre eux ou avec l'extérieur de manière sécurisée.

Lister les réseaux : `docker network ls`

1. **Le réseau par défaut : bridge** Quand on lance un conteneur sans spécifier de réseau, Docker le place dans le réseau bridge par défaut.

Inspection : `docker network inspect bridge` (Affiche la liste des conteneurs qui l'utilisent).

2. **Le réseau hôte : host** Principe : Docker n'attribue pas d'IP spécifique au conteneur. Il n'y a aucune isolation réseau. Le conteneur utilise directement le réseau de la machine hôte.

Avantage : Pas besoin de mapper les ports (`-p`).

```
docker run --network host nginx
```

3. **Le réseau isolé : none** Principe : Le conteneur n'a aucune carte réseau, ni accès à internet, ni accès aux autres conteneurs. Il est totalement en quarantaine.

Cas d'usage : Traitement de données ultra-sensibles, exécution de scripts non fiables.

```
docker run --network none -it alpine sh
# Si on fait un "ping google.com" à l'intérieur, cela échouera (Network unreachable).
```

4. **Le réseau personnalisé (User-Defined Bridge)** Si l'on souhaite regrouper des conteneurs (ex: un serveur web et une API) pour qu'ils communiquent entre eux de manière sécurisée, on crée un réseau dédié.

Avantage : Permet la résolution de noms (DNS). Les conteneurs peuvent se "ping" en utilisant leur nom (`--name`) au lieu de leur adresse IP.

Création et utilisation :

```
# 1. Créer le réseau personnalisé
docker network create monreseau

# 2. Lancer des conteneurs dans ce réseau
docker run -d --name site --network monreseau nginx
docker run -d --name api --network monreseau alpine sleep 1000
```

Autres commandes de gestion : `docker network connect monreseau site` : Ajouter un conteneur existant à un réseau. `docker network disconnect monreseau site` : Retirer un conteneur d'un réseau.

12. Automatisation avec le Dockerfile

Jusqu'ici, la configuration des conteneurs (installation de paquets, création de fichiers) se faisait "à la main" en mode interactif. Le Dockerfile permet d'automatiser tout ce processus dans un fichier texte pour créer une image réutilisable, prête à être partagée (sur Docker Hub par exemple) et déployée sur n'importe quelle machine.

Les Instructions Principales

Instruction	Rôle
FROM	Choisir l'image de base (doit toujours être la première instruction).
RUN	Exécuter une commande à l'intérieur de l'image (ex: <code>apk add curl</code>) lors de sa construction.
COPY	Copier un fichier ou dossier de la machine hôte locale vers l'image.
WORKDIR / SOAP	Définir le dossier de travail par défaut à l'intérieur de l'image.
EXPOSE	Documenter le port exposé par l'application (informatif).
CMD / ENTRYPOINT	La commande qui sera lancée au démarrage du conteneur.

Exemple 1 : Image Alpine simple Création d'une image qui affiche un message au démarrage.

Fichier Dockerfile :

```
# Utilise une image Alpine légère
FROM alpine:latest

# Définit la commande exécutée au démarrage du conteneur
CMD echo "Bonjour Polytech Nancy"
```

Construction (Build) et Exécution :

```
# L'option -t permet de nommer (tagger) l'image.
# Le "." indique à Docker de chercher le Dockerfile dans le répertoire courant.
docker build -t mon-app .

# Lancement du conteneur basé sur notre nouvelle image
docker run mon-app
```

Exemple 2 : Application Java (Multi-Stage Build) Si nous voulons exécuter une application Java, il serait mauvais d'intégrer le code source (Hello.java) et le compilateur (javac) dans l'image finale destinée à la production, car cela l'alourdirait inutilement. On utilise alors des étapes (Multi-stage) : une pour compiler, une pour exécuter.

Code Java (Hello.java) :

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Bonjour Polytech Nancy");
    }
}
```

CM du 10/04 :

14. Scalabilité et Optimisation des Images

Pour qu'une application puisse "passer à l'échelle" (scalabilité), elle doit garantir un temps d'exécution constant, quel que soit le nombre d'utilisateurs simultanés.

- **Le défi** : Lors d'un pic d'utilisation, le système doit anticiper et démarrer de nouvelles instances (conteneurs) très rapidement.
- **La solution** : Les images Docker doivent être les plus légères possibles. Une image trop lourde mettra trop de temps à se télécharger et à s'instancier, ralentissant la réponse au pic de charge.

*L'image Docker doit être la plus légère possible (d'où l'utilisation de versions slim ou alpine). Une image lourde met trop de temps à se télécharger et à démarrer. *

15. Frameworks et ORM (Object Relational Mapping)

- **Rappel Flask** : Flask est un framework. Sans lui, il faudrait écrire toute la logique de routage HTTP à la main.
- **ORM (Object Relational Mapping)** : C'est un outil qui fait le pont entre la programmation orientée objet (Python, Java) et les bases de données relationnelles (SQL).
 - *Principe* : On crée une Classe en Python, et l'ORM se charge de la sauvegarder dans la base de données.
 - *Fonctionnement* : Utilise souvent des annotations (ex: @Entity, @OneToOne). C'est le framework qui gère automatiquement la couche d'accès aux données (DAL - Data Access Layer).

16. Limites du Dockerfile et Introduction à Docker Compose

- **Le Dockerfile** : Conçu pour exécuter **une seule application / un seul service** à la fois. Si l'API REST (ex: `app.py`) a besoin d'une base de données, un Dockerfile seul ne suffit pas.
- **Docker Compose** : L'outil standard pour les environnements multi-services (ex: *une API REST + une Base de données*). Il permet de démarrer l'ensemble et de les faire communiquer sur un réseau virtuel.
 - *Principe* : Décrit toute l'architecture d'un projet dans un seul fichier : `docker-compose.yml`.
 - *Avantages* :
 - Une seule commande pour tout démarrer : `docker compose up`.
 - Les conteneurs communiquent automatiquement entre eux via un réseau virtuel interne.
 - Les configurations sont versionnées, reproductibles et partageables.
 - Idéal pour lancer des stacks complètes (Base de données + API + Frontend + Cache).

17. Introduction aux Bases de Données Distribuées et Kubernetes

Spécificités des BDD Distribuées : Contrairement à une base de données relationnelle classique, les données envoyées sont des requêtes exactes (problème de polysémie, exemple de l'avocat (cf Projet TAL)).

Tolérance aux pannes (systèmes distribués) : Si une panne survient au milieu du processus, elle ne doit pas impacter la recherche ou la transaction en cours. Ce mécanisme est possible car la gestion reste locale grâce à la fragmentation des données.

18. Exemple Pratique d'Architecture Multi-Conteneurs

Pour illustrer les dépendances entre services :

- Service 1 (Backend) : Une API REST développée en Python qui renvoie une liste de produits.
- Service 2 (Frontend) : Une application PHP qui consomme cette API pour afficher la liste des produits.

Note : L'application PHP est ici strictement dépendante de l'API REST Python.

19. Sprints

N.B : À faire -> Check les vidéos de référence pour la partie dédiée à l'organisation en Sprints.

20. Kubernetes

Si Docker Compose est suffisant pour notre environnement local, ce qui va véritablement permettre de créer, gérer et maintenir plusieurs conteneurs à grande échelle et en production, c'est Kubernetes.

N.B : À faire -> Check Kubernetes.

